Did the pandemic and e-bikes change how New Yorkers use Citi Bike?

A Medallion pipeline over 334 million trips (2013–2026)

Alexander Rebello
DAT535 final project
Code: `https://github.com/alexander-rebello/citibike-medallion`
Dashboard: `https://alexander-rebello.github.io/citibike-medallion/`

November 2026

1 Question and consumer

Citi Bike, New York's bike-share system, publishes every trip since June 2013. Two events changed the system after 2019: the COVID-19 pandemic in 2020 and the switch to an electric fleet, which today carries most trips. The question for this project is whether these events changed *how* New Yorkers use the system. The consumer is a city planner or the operator, so the answer has to be traceable to the raw data. Four measures were fixed with the supervisor before any code was written: (1) share of casual riders, (2) daily demand peaks, (3) trip length on e-bikes vs classic bikes, and (4) whether the busiest start points move away from central Manhattan. The core comparison is 2015 / 2019 / 2024; the pipeline processes the full history (June 2013 to October 2026) so that the core years can be seen in context.

The answer (Section 9) depends on Silver: it is only as good as the cleaning of 334 million rows written by two generations of software with eight timestamp formats. Silver therefore takes the most space in this report.

2 Understanding the raw data

The source is a public S3 bucket: yearly zip files for 2013–2024 (some with nested folders and macOS metadata files) and monthly zips from 2025. Jersey City files are out of scope. A downloader streams every CSV into gzip without changing a byte and writes a manifest (file, month, size, header, row count). In total there are **161 monthly datasets with** 333,888,858 **rows**.

Two schemas, four headers. The data has two logical schemas. `legacy_v1` (2013-06 to 2019-12) has a trip duration, bike id, birth year and gender, and one coordinate pair per station. `modern_v2` (from 2020) has a ride id, bike type (classic/electric), member/casual, and coordinates *per trip* with 13+ decimals, but no bike id or demographics. Legacy appears with three header spellings (lower case, quoted, and Title Case for 2016-10 to 2017-03). Every header is identified by a fingerprint (first 16 hex digits of SHA-256 over the lower-cased header) in a schema registry; a header that is not registered stops the pipeline (Section 8).

Duplicated file sets. 19 yearly zips contain the same month twice (one whole file and the same data split into parts). I proved for each that the row counts are equal and kept one set; otherwise these months would have been counted twice.

A wrong assumption, corrected by measuring. An early sample of January and June files suggested one timestamp format per year. Before writing any Silver rule I profiled all 161 months on the cluster (7 min 40 s): every value of every column was classified by its shape and counted per month. The result (Figure 1) is eight runs of formats that change by month, never within a month. 2015 alternates between formats with and without seconds (January–March and June have no seconds), which the January/June sample could not reveal: no single pattern fits the whole year, so a rule per year would have failed for part of 2015. The same profile found the station-id styles, the birth-year markers, the service stations and the coordinate outliers that drive the Silver rules in Table 1.

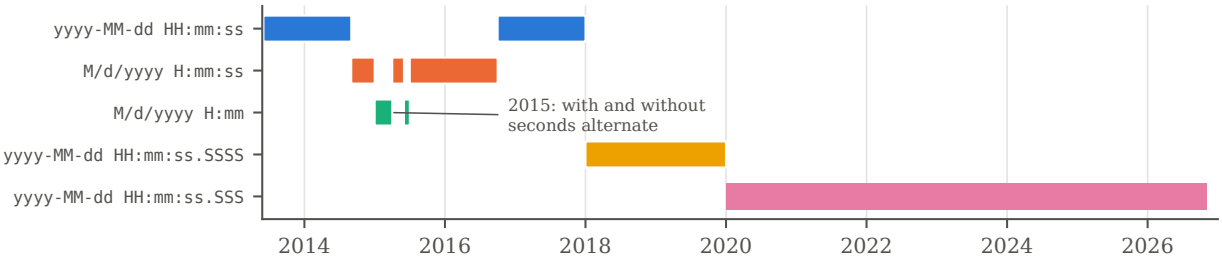


Figure 1: Timestamp format by month, measured over all 161 months. Each format run gets its own parse pattern in `config/silver_rules.toml`; months without seconds take the duration from the `tripduration` column.

3 Architecture and environments

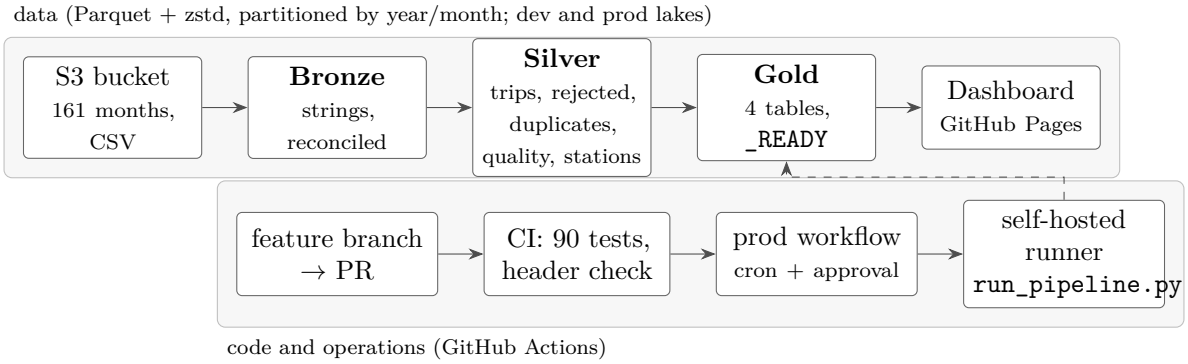


Figure 2: Architecture. Every layer is Parquet on a shared file system; Spark runs on a two-node standalone cluster.

The pipeline is a Medallion architecture on files (Figure 2). Each layer is a set of Parquet tables (zstd, partitioned by year and month) plus a ledger and a run log in `_meta/`. All transformations use the Spark DataFrame API and Spark SQL; there are no Python UDFs.

Dev and prod. One variable, `CITIBIKE_ENV`, selects a configuration file and a separate lake. Dev holds six months chosen so that every schema, header spelling and timestamp format appears at least once; a full dev run takes about two minutes. Prod holds all 161 months (Bronze 9.6 GB, Silver 14 GB, Gold 0.36 GB).

Cluster. Two virtual machines with 8 vCPU and 16 GB each. The main node holds the data, the Spark master, one worker, the driver and the GitHub Actions runner; the second node runs a worker and mounts the lake over NFS v4.2 at the same path. Master and workers run as systemd services,

configured from the repository (`infra/spark/`), so the cluster survives reboots and is rebuilt the same way each time. Every job writes a Spark event log, which is the evidence for Section 7.

4 Bronze: store unchanged, reconcile every month

Bronze keeps the source as it is, in a form Spark can read fast. Decisions:

- **All-string schema, given explicitly**, plus `_corrupt_record`. Nothing is inferred and nothing is converted, so no value can be lost before Silver has looked at it. A wrong field count keeps the row with its raw text.
- **Refuse instead of guess.** A month is only loaded if its source selection is unambiguous and every header is in the registry; otherwise it is refused with a reason.
- **Reconcile.** After writing, the rows per month are compared with the manifest. Only reconciled months enter the ledger, and Silver reads only months in the ledger. Spark silently skips blank lines; reconciliation would expose them as a difference.
- **Idempotent.** Each month's ledger entry stores a digest of its source files; re-running skips unchanged months and re-loads changed ones.

Result on prod: 161 months loaded, 0 refused, 0 failed, every row count equal to the manifest, 0 corrupt records. Bronze adds metadata columns (`_source_file`, `_schema_version`, `_header_fingerprint`, `_run_id`, `_ingested_at`) so that every Silver row can be traced back to its file and load.

5 Silver: from 334 million raw rows to trustworthy trips

Silver maps both schemas onto one set of trip columns, applies quality rules, removes duplicates and attaches a station key that is stable across the two eras. Three principles guided it:

1. **Rules come from measurements.** Every rule answers a finding of the profile and its effect was measured on all 161 months before Silver was written (`silver_rules_report.py`).
2. **No row disappears without a reason.** A row is either accepted, rejected (with all rules it breaks, the first being its `reject_reason`) or removed as a duplicate (with the reason and the copy that was kept).
3. **Flag what is real but special; reject only what is not a rider trip.**

5.1 Rules

The rules live in `config/silver_rules.toml`, not in code; changing that file makes Silver re-process every month.

5.2 Row accounting

Every month must satisfy *Bronze rows = accepted + rejected + duplicates removed* before it is published. Table 2 shows the result for the full history.

Three observations from the per-year impact: (1) the >24 h rule is the largest in every era and peaks in 2022 (40,185 rows); (2) the <60 s rule barely bites (579 rows) because the operator already removes very short trips; (3) missing start stations were almost absent until 2022 and then grew to 0.04–0.07 % of modern trips. The rejected rows are kept with all their reasons, so a planner who disagrees with a threshold can see exactly what it removed.

5.3 Daylight saving: a bug in the data, then a bug in my repair

Before the repair, 1,760 modern trips had a duration ≤ 0, and *all* of them were in November of 2021–2025. Modern timestamps are local time without an offset. On the night the clocks go back,

Rule	Problem found in the data	Silver decision and evidence
Timestamps	8 format runs (Figure 1); 4 months without seconds	One pattern per run; unknown values become null and are rejected (`time_missing`): **0 rows** in 161 months.
Station ids	Legacy 2018/19: 46 %/38 % written as `3160.0`; modern `5303.06_`	Ids stay strings; strip `.0` and trailing `_` before matching.
Rider type	Legacy Subscriber/Customer (0.1–0.3 % empty), modern member/casual	Map to member/casual; empty stays unknown, not guessed.
Birth year	`\N` (2013–14), empty (2015–17), `1960.0` style, values before 1920	Parse; null implausible values, flag `birth_year_nulled` (41,807).
Trip id	Legacy has none	Hash of start, stop, bike and start station: the same trip always gets the same id, so duplicates can be found.
Duration	Legacy has 0 trips < 60 s (source pre-filters); up to 44 days	Reject < 60 s and > 24 h.
Service stations	Depots, workshops, test points (`SYS016`, `NYCBS Depot`)	Reject (`service_station`); 55 matched id/name pairs, all checked by hand.
Valet, Jersey City	Staffed parking (Penn Station Valet); JC*/HB* ids in NYC files	Keep, flag `is_valet`, `outside_nyc`; excluded from station ranking only.
Coordinates	Montreal "8D" stations, $(0, 0)$ depots	Reject outside the NYC box.
End station	0.25–0.35 % of modern trips have none	Keep, flag `is_incomplete`.
Daylight saving	Local time without offset (Section 5.3)	Repair 2,431 trips, flag `dst_repaired`.

Table 1: Silver rules: each answers a measured problem. Full per-year impact in `docs/silver_rules_impact.md`.

Row accounting (161 months)			Rejected by first reason	Rows
Bronze	333,888,858	100 %	duration > 24 h	158,950
Accepted (`silver/trips`)	333,588,780	99.91 %	no start station	94,360
Rejected (`silver/rejected`)	299,547	0.09 %	service station	45,608
Duplicates removed	531	$<$0.001 %	duration < 60 s	579
Accepted without station key	0		outside coordinates	50
			time missing, duration ≤ 0	0

Table 2: Silver result on prod. Both sides balance per month; a month that does not balance is not published.

01:00–02:00 happens twice, so a trip that starts at 01:50 in the first pass and ends at 01:05 in the second reads as ending before it starts. The repair adds one hour to the end time of such trips. Legacy data has a second source of truth, the `tripduration` column: 671 legacy trips disagreed with their timestamps by exactly 3,600 s around the change, and their end time is rebuilt from start plus duration. After the repair no trip has a duration ≤ 0, and only 13 legacy trips (2013–14) still disagree with `tripduration`; they are flagged, not changed.

My first version derived the `dst_repaired` flag *after* the end time had been changed, so it tested a repaired value and gave wrong results. The fix was to decide once per row and store the decision in a column, covered by four tests with hand-made November trips; I now apply this to every derived flag. **Limitation:** a modern trip that ends in the second pass but still reads as after its start (e.g. 00:50 $\rightarrow$ 01:10) cannot be detected from the data and keeps a duration one hour too short.

5.4 Duplicates

Three kinds are handled, each with its own reason in `silver/duplicates`: the same trip id in two files (keep the earlier file's copy), a trip id of a new month that is already in the published previous

month (`cross_month_prior`, needed for monthly runs), and in legacy data one bike starting twice at the same moment (keep the shorter trip). The result was small but real: 512 modern trips started on 30 April 2026 and ended after midnight, so they appear in both the April and the May file (483 identical, 29 differing only in the last digit of a coordinate), plus 19 legacy rows. For efficiency the candidate keys are found from the key columns only; just the few hundred affected rows are ranked with a window function, instead of ranking all 334 million.

5.5 Station dimension across two id systems

Legacy ids (`455`) and modern ids (`6584.12`) are unrelated, names change, and some ids are reused for a different place. Questions about busiest stations need one station key across 13 years. I measured first: of 1,086 names present in both eras, 1,076 lie within 50 m, one is more than 1 km apart (a reused name); six legacy ids were reused for a place more than 1 km away. From this the rule follows: two records (schema, id, name) are the same station if they share an id or an exact name *and* lie within 50 m (a missing position cannot contradict); chains are followed as connected components (min-label propagation in Spark); stations that exist in only one era are matched by position only if the match is mutual and unique. Pairs that share an id or name but are more than 50 m apart are never linked; all 44 are listed in `station_conflicts`. Positions are medians of trip coordinates over 10 m cells, and the key is a hash of the earliest record's label, so it does not change when months are added.

Result: 4,205 records → **2,837 stations**, 967 of them in both eras (78 % of all trip occurrences). I hand-checked all 7 position-only matches (6 correct or plausible renames, 1 doubtful: a temporary event station with 300 trips) and a random sample of 10 id/name matches (all correct). One rule was added after a rebuild merged a valet record into the regular station next to it, which would have removed a busy station from the ranking: valet records are never merged with regular stations.

5.6 Incremental runs and checks

A month is re-processed only if its Bronze digest, the previous month's digest (cross-month duplicates), the rules file, or *its own* stations changed (a per-month fingerprint of the station keys it uses). Stations are updated incrementally: only new or changed Bronze months are scanned. All months of a run are written once to `silver/_staging/<run_id>`; each month is checked (row accounting, every accepted trip has a start station key) and only passing months are published and written to the ledger. A failed month leaves the staging area for inspection.

Tests. 90 tests run in CI on small hand-made fixtures for both eras. They include the cases that went wrong once: each timestamp format, `3160.0` ids, November trips around the clock change, the valet merge and a duplicate across months.

6 Gold: tables that answer the question

Gold builds four tables from accepted Silver trips only: `gold_riders_daily` (trips and mean duration per day, rider type and bike type), `gold_demand_hourly` (trips per start station, day and hour), `gold_station_year` (trips per station and year, rank among regular NYC stations, distance from Midtown) and `gold_station_bands` (trips by distance band from Times Square: 0–2, 2–5, 5–10, 10+ km; valet and Jersey City separately). Legacy trips have no bike type and are counted as classic, which was true at the time.

Gold is published only if it is consistent with Silver. A run builds all tables in `gold/_staging/<run_id>`, checks that every table's total equals the accepted Silver count and that the Silver ledger shows every month published and balanced, then swaps the tables in and writes `gold/_READY` with the run id and the checks. A failed check leaves the previous Gold and its `_READY` untouched. The dashboard reads

only Gold and refuses to build without `_READY`. Current publication: run `20261109T233831Z-2f17e7`, 333,588,780 trips, all checks passed.

7 Performance, backed by evidence

I changed the configuration only when an event log showed a reason, and measured before and after. Table 3 summarises the experiments, including the hypotheses that turned out to be wrong.

Experiment	Evidence that led to it	Result	Verdict
Bronze: 1 vs 2 nodes		8:39 → 4:57 (**1.75**×); tasks 257/258. Hypothesis "second node is slower because it reads over NFS": median task 11.1 s vs 11.4 s. CPU-bound; ∼33 s fixed overhead.	speed-up; NFS hypothesis **refuted**
Station scan (667 M occurrences)	14:33 for a scan that should cost like Bronze	Fix 1, aggregate to 10 m cells first: 13:48, only 8 % better. Event log: the three largest stages each read the same 2.3 GB; the grouped table fed three aggregations and was recomputed each time. Fix 2, `localCheckpoint` on the small grouped table: **2:14**.	fix 1 **refuted**; fix 2 **6.5**×
Silver: sort before partitioned write	83 GB spill in a job without a shuffle	Spark 3.4+ plans a sort by the partition columns before writing. Disabled (`plannedWrite.enabled=false`, 8 open writers): run 280 s → 253 s (**−10 %**), publish job −36 %, spill 14 GB → 0. The main transform job (137 s) did not change: it is CPU-bound by parsing and rules.	smaller win than expected
Gold: small files	10,304 files for 598 MB	`repartition(year, month)` before writing: build −15 %, read −17 % (test lake); prod: **161 files, 356 MB**, Gold step 111 s → 80.5 s.	confirmed at full scale

Table 3: Performance experiments. Silver and Gold tests on a separate experiment lake (2019 + 2024, 64.8 M rows), runs alternated and caches dropped on both nodes before each run, $n = 2$; Bronze and station scan $n = 1$.

The largest gain did not come from tuning a parameter but from reading the event log: the expensive part of the station scan was not the work per row but how often Spark repeated the scan. The same pattern (a DataFrame reused by several actions) is now checked in every new job. The 1.75× speed-up from the second node is below 2× because of the fixed overhead and because the driver, NFS server and runner share the main node. The Silver experiment shows the opposite of a win by assumption: I expected the sort to dominate the whole Silver job; it only dominated the publish step.

8 Operations: deployment, schedule, failure handling

- **Change process.** All work went through 17 feature branches and pull requests. CI (`ci.yml`) runs the 90 tests with a local Spark session and checks the source headers against the registry; a pull request is merged only when it is green.
- **Prod workflow** (`prod.yml`) runs on the self-hosted runner, monthly by cron (06:00 UTC on the 5th, when last month's file is published) or by hand with inputs (months, force, cluster). It uses the GitHub environment `prod` with a required reviewer, so every run waits for approval before it touches the prod lake, and a concurrency group so that two runs never overlap. It uploads

the step logs as an artifact, writes a summary to the run page and, if everything succeeded, publishes the dashboard to GitHub Pages.

- **Pipeline runner** (`run_pipeline.py`) runs the steps in order (`bronze.download`, `bronze.select`, `bronze.ingest`, `silver.stations`, `silver.trips`, `gold.tables`, `gold.dashboard`), stops at the first failure, keeps one log per step and appends to `pipeline_runs.jsonl`. A run with no new data takes about 3 minutes, most of it the full Gold rebuild (80 s).
- **Failure demo** (`failure-demo.yml`) injects a source with an unregistered header. The run stops at source selection with exit code 1, the later steps are skipped and Gold's `_READY` is unchanged (run 42653700366).

9 Results

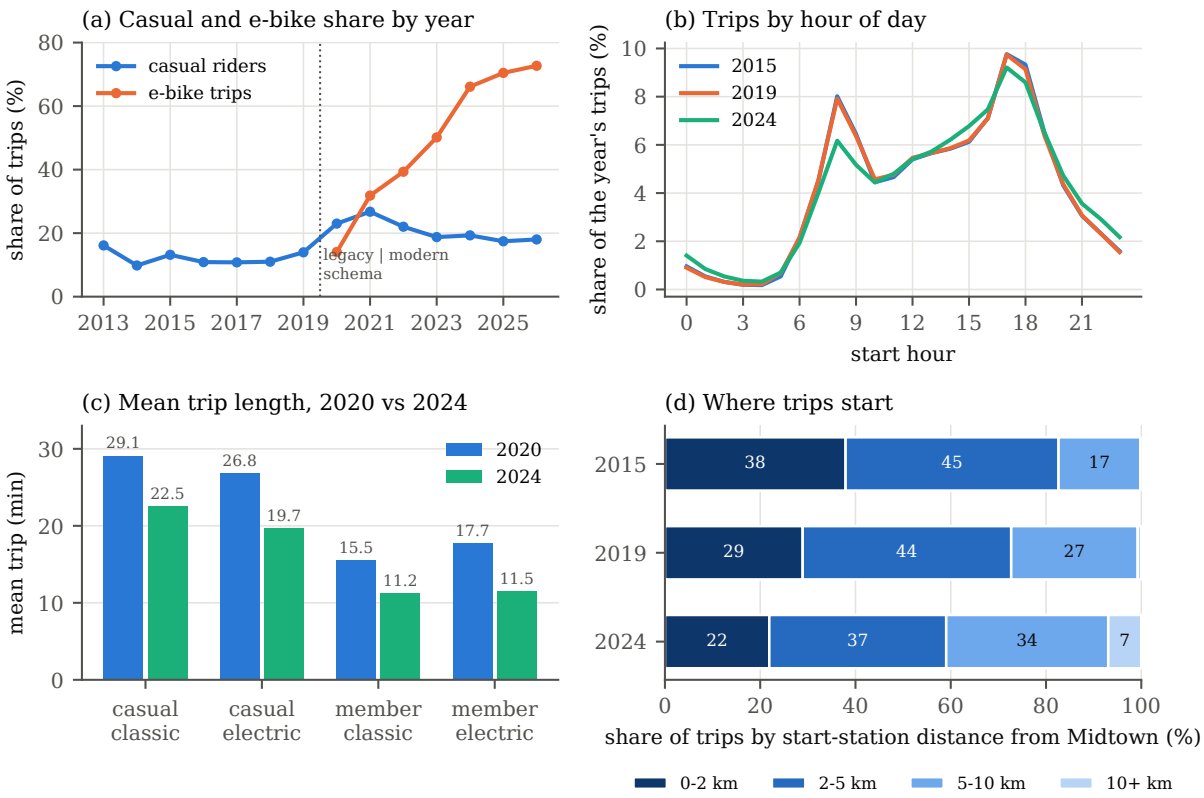


Figure 3: Answers from Gold (run `20261109T233831Z-2f17e7`). (a) Before 2020 "casual" is the legacy *Customer* type, after it the modern *casual* type. (d) Distance of the start station from Times Square.

Casual riders. The casual share was 13.2 % in 2015 and 14.0 % in 2019, jumped to 23.0 % in 2020 and peaked at 26.7 % in 2021, then fell back to 19.3 % in 2024 (Figure 3a). The pandemic brought a lasting increase, but half of the jump has faded. The definitions change between the schemas, so the 2019→2020 step is partly a definition effect; the 2020–2024 trend is within one definition.

E-bikes. The electric share grew from 14 % (2020) to 66 % (2024) and 73 % (2026); since 2023 most trips are electric. Mean trip length rose from 14.3 min (2019) to 18.8 min (2020) and fell to 13.2 min (2024). The e-bike effect differs by rider type (Figure 3c): in 2024 casual riders are about 3 minutes faster on an e-bike (19.7 vs 22.5 min), members ride equally long on both (11.5 vs 11.2 min), so members seem to use the e-bike to go farther, not to arrive earlier.

Daily peaks. The evening peak at 17:00 is stable. The morning commute peak weakened: 8:00 carried 8.0 % of trips in 2015, 7.9 % in 2019 and 6.2 % in 2024, while the early afternoon and evening hours gained (Figure 3b). This fits more flexible work after the pandemic.

Away from central Manhattan. The share of trips that start within 2 km of Midtown fell from 38 % (2015) to 29 % (2019) and 22 % (2024); trips starting more than 5 km away grew from 17 % to 41 % (Figure 3d). The top-five stations, however, are still within 3 km of Midtown in all three years except West St & Chambers St (5.1 km, 2024). Most of the shift is the network's growth into Brooklyn, Queens and the Bronx (471 start stations in 2015, more than 2,200 in 2024), so it measures where the system reaches more than a change in behaviour.

Answer: yes, both events changed use. The pandemic left a higher casual share and a weaker morning commute; e-bikes became the default bike within four years; usage spread out from central Manhattan, mostly because the network did.

10 Limitations and reflection

Limitations. Legacy data has no bike type (counted as classic) and a different rider definition. One daylight-saving case cannot be detected (Section 5.3). The station shift is confounded by network growth; a fair comparison would restrict to stations that existed in all three years. Performance numbers come from one or two runs on shared VMs; the Silver write change was not repeated on the full history. The NFS server and driver share one node, a single point of failure.

What I would do differently. (1) Profile everything before writing rules: one sample cost me a set of wrong timestamp rules, and the full profile took less than 8 minutes. (2) Store every decision in a column at the moment it is made; deriving it later from changed data caused the daylight-saving flag bug. (3) Read the event log before forming a hypothesis: two of my four performance hypotheses were wrong, and the event log showed why each time. (4) Accounting checks in every layer (Bronze vs manifest, Silver balance, Gold vs Silver) were the most useful quality measure: they turned "looks right" into a check that fails the run.

With more time. Compare only stations that exist in all core years; join weather data (NOAA Central Park) to separate weather from behaviour in the yearly trends; build Gold incrementally instead of fully each month.